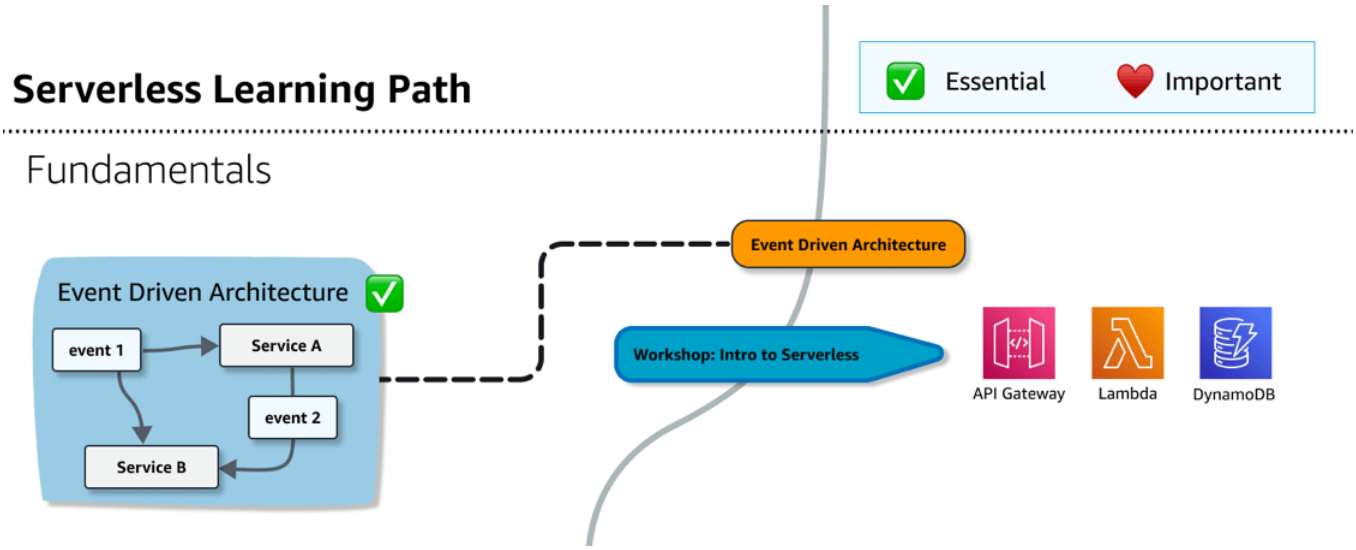
Transitioning to event-driven architecture

Event-driven architecture (EDA) is the first step on the serverless learning path. Understanding how services interact through *events* is essential to successful serverless development.

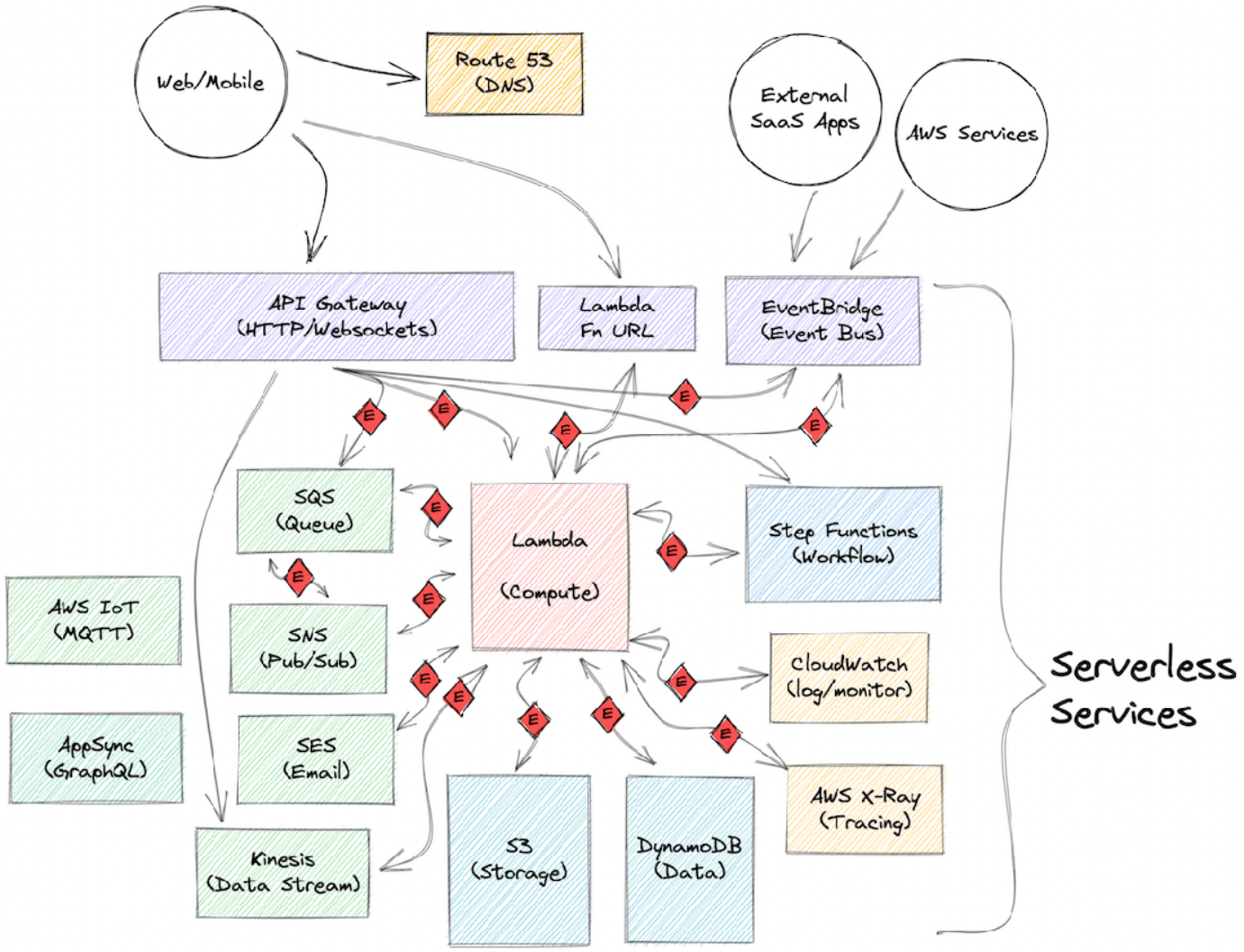
In this chapter, you will dive into the transition from traditional to event-driven architecture.



Let's start by thinking about event-driven components of a food delivery service.

You'll need a data store for menus, locations, orders, status. Customers will send network requests to a web API. Your application then needs a compute resource to process customer orders.

You can handle long-running tasks asynchronously. For example you can implement a queue using Amazon SQS to manage order submission on. You can then use Step Functions to manage a workflow that updates user information, and inventory counts after every order is processed. Along the way, you will need to log actions, monitor app activity, and trace data flows to debug.

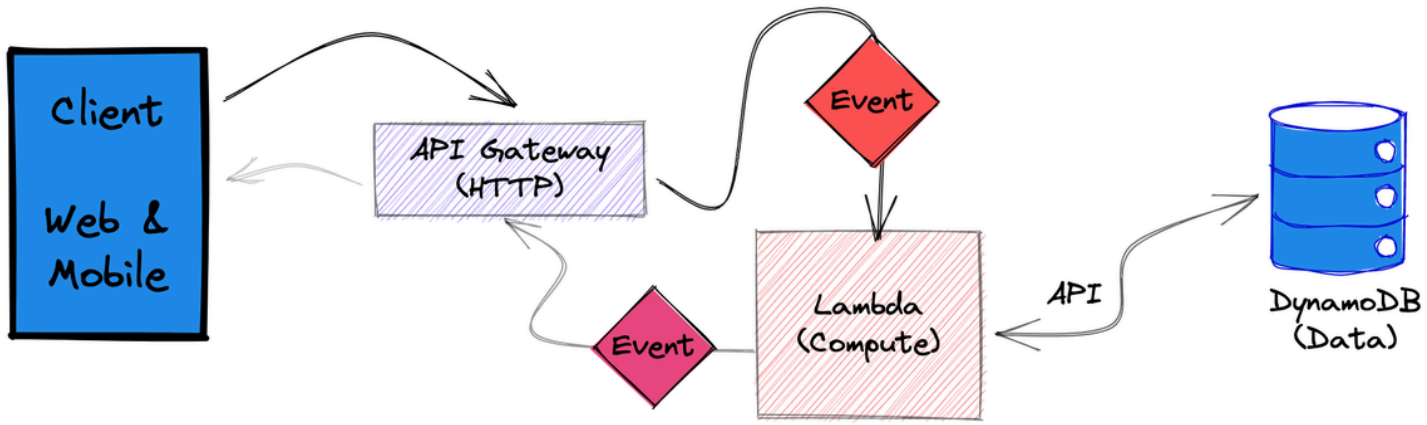


If you're new to serverless development, you might be more familiar with traditional frameworks. Let us look at the steps in a traditional request/response cycle for comparison:

1. Accept an inbound network request and create local data objects.
2. Map the URL, often called a route, with configuration or annotations to an action.
3. Create global data and utility services, such as a database connection pool and an object relational mapper.
4. Implement web hooks for logging, observability, and health monitoring components.
5. Process the request: query a database, store or retrieve data, call external systems.
6. Convert outbound data into a suitable response or error and serialize the data into JSON.
7. Add metadata, such as headers, cookies, tokens to the response object.
8. Send the response back to the client.

Now, let us compare a similar work flow implemented with event-driven architecture.

This diagram represents a microservice that retrieves data from a database, for example, retrieving shopping cart items for a customer order.



First, a web or mobile client makes an HTTP request to `GET /cart/`*`A1234B56`* for a list items in a cart.

A component needs to accept the request and extract metadata, such as HTTP method, path, extra path info, query string parameters, headers, and cookies. That component will also verify the request is from an authenticated and authorized entity. In AWS, API Gateway accepts the inbound URL, extracts the parameters, query string, and headers and creates an event to send to other services for processing.

Example based on event.json API Gateway proxy event for a REST API:

```
{
  "resource": "/cart",
  "path": "/cart/A1234B56",
  "httpMethod": "GET",
  "headers": {
      "accept": "text/html,application/xhtml+xml,application/xml;q=0.9",
      "accept-encoding": "gzip, deflate, br",
      "User-Agent": "Chrome/80.0.3987.132 Safari/537.36",
      "X-Amzn-Trace-Id": "Root=1-5e66d96f-7491f09x5pl79d18acf3d050", ...
  },
  "multiValueHeaders": {
      "accept": [
          "text/html,application/xhtml+xml,application/xml;q=0.9,"
      ],
```

```
      "accept-encoding": [
          "gzip, deflate, br"
      ],

      ...

      "queryStringParameters": null,
      "multiValueQueryStringParameters": null,
      "pathParameters": null,
      "stageVariables": null,
      "body": null,
      "isBase64Encoded": false
  }
}
```

Tip

A dedicated API service might at first seem unnecessary, but implementing this action as a separate service allows flexibility and scalability to your solution. For more information about working with API Gateway, see the the section called “API Gateway”.

An *event* represents a change in state, or an update.

For example: an item placed in a shopping cart, a file uploaded to a storage system, or an order becoming ready to ship. Events can either carry the state, such as: quantity (qty), item price (itemPx), and currency; or simply contain *identifiers* needed to look up related information, such as: customerId and orderId, as shown in the following example of a `NewOrderEvent`:

Event name

NewOrderEvent

```
1  {
2    "source": "myApplication",
3    "detail": "submitOrder",
4    "customerId": "customer123",
5    "orderId": "order-A1234B56",
6    "paymentStatus": "open",
7    "cart": [
8      {
9        "product12": {
10         "qty": 2,
11         "itemPx": 35.22,
12         "currency": "USD"
13       },
14       "product44": {
15         "qty": 5,
16         "itemPx": 71.57,
17         "currency": "USD"
18       }
19     }
20   ],
21   "timestamp": 1607774286
22 }
```

Next, API Gateway integrates with Lambda, a compute service, to handle the new event. Lambda function code parses the parameters in the inbound event, connects to the data store, and retrieves the cart. The function queries the database API through an SDK library. Because the DynamoDB database is also serverless and built to respond with low latency, there is no need for a connection pool.

After converting currency to USD and removing unavailable items, the function sends the result as a new event to API Gateway.

Finally, API Gateway converts the event into a response to send to the waiting client.

The method with which a function is invoked should be informed by your application architecture and needs. For example, batch-processing patterns have different applications to on-demand data processing. Understanding these paradigm differences can also help customers decide between AWS services.

Deploying a microservice as a containerized application on Fargate could be more appropriate if the microservice is primarily used for batch data processing. Whereas a Lambda function would be much more straight-forward to deploy and maintain in applications that require on-demand data processing.

Decoupled event-driven architecture

For simpler applications, the advantage of event-driven versus request-driven applications may not be apparent. But, as your applications add more functionality and handle more traffic, the value becomes clear.

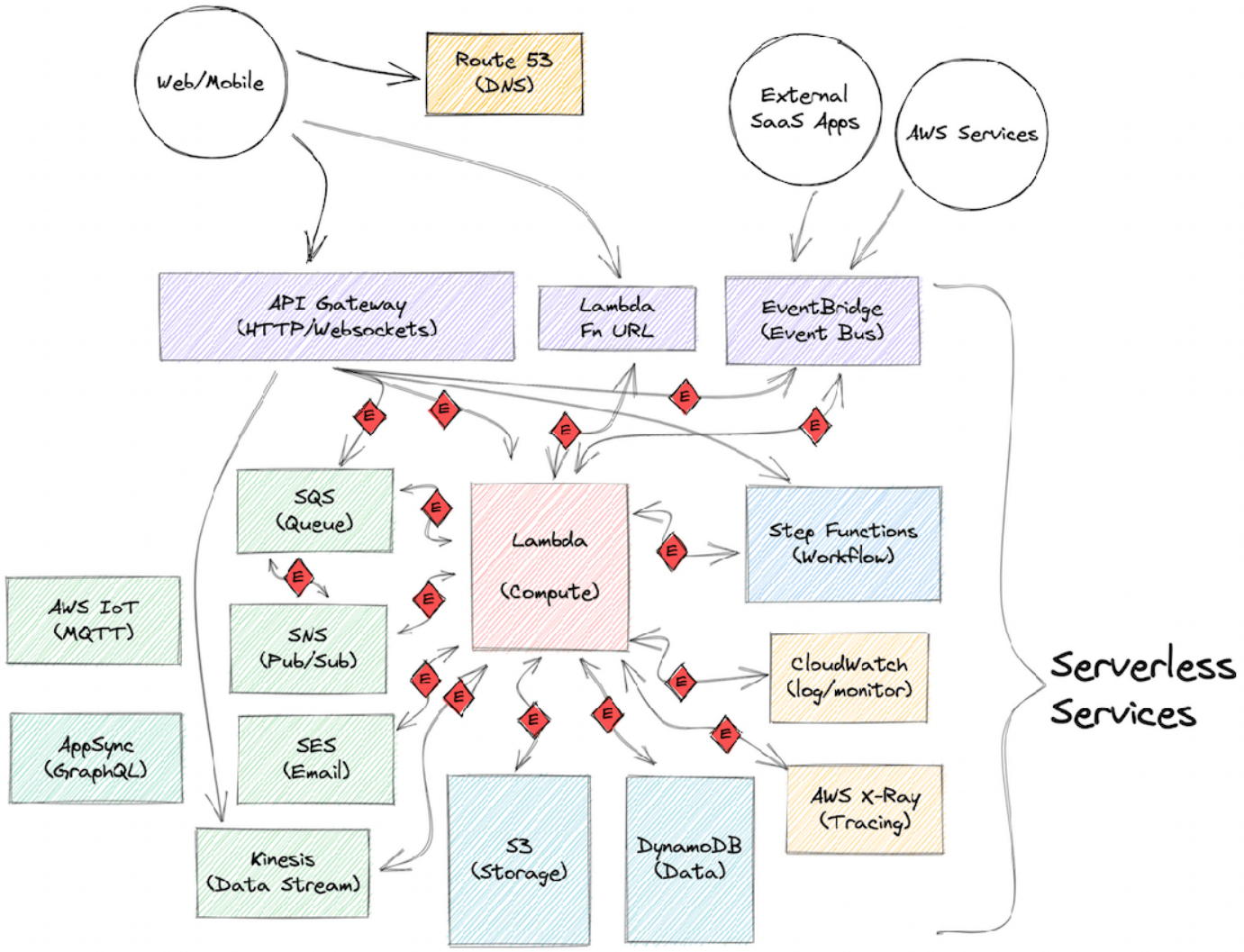
Event-driven applications rely on communication through events that are also observable by other services and systems. Event producers are unaware of which (if any) consumers are listening. This strategy makes it easier to extend and scale, without disrupting existing work flows.

For example, API Gateway can enforce rate and volume limits for requests to your API on a per-customer basis. Or, a service could watch the inbound query parameters to create a list of popular product searches. The database could send events when products are added to the cart, to feed to a predictive ML algorithm for ordering supplies.

Due to the loose coupling between components of an event-driven system, your compute functions are not even aware of these other activities. You can scale components independently. One service can fail, without impacting other services. Events can be flexibly routed, buffered, and provide a log for audit.

Let’s revisit the diagram with various services connected through events. We can start to see now that it is actually not as complex as it might have appeared.

Think of it like looking down on a big city with messengers moving packages and letters between people and businesses. The sources and destinations range from the suburbs to the city core. Inbound requests could be managed by a dispatcher, like API Gateway, Lambda Function URLs, or Amazon EventBridge. Or, some messages might be dropped in a box for asynchronous delivery. This would be like events routed to queues or orchestrated in complex work flows with Step Functions. The monitoring, tracing, and metrics services, CloudWatch and AWS X-Ray, are like managers, watching the stream of events to make sure packages are delivered, and if not, to troubleshoot the problem. After drop-off at a business, some packages are transferred by in-house delivery agents to their final destination. This situation is similar to how services that store data and files, may stream events to compute services, which triggers ever more actions.



Connecting services with **events** and **event-driven architecture** gives you a consistent and scalable way to build solutions with hundreds of services.

Summary

- Serverless is built on independent services that communicate through *events* in an *event-driven architecture.*
- *event-driven architecture* (EDA) - a modern architecture pattern built from small decoupled services that publish, consume, or route *events.*
- *events* - represent a change in state, or an update
- Decoupled microservice architecture helps you build modern, agile, and extendable applications faster than traditional monolithic applications, and free developers from needing to learn everything at once about existing systems.

Next steps

- See What is EDA? for advantages of a decoupled architecture.
- Learn more about the advantages of modernizing monolithic applications in the AWS Prescriptive Guidance enabling data persistence in microservices reference document.